

AlGroove — 관리자 백오피스 시스템

신입 백엔드 개발자 포트폴리오 · 김형준

huneig@naver.com

개인 리팩토링 저장소 github.com/kimhyeongjun-1204/aigroove · 원본 팀 저장소 github.com/mys990930/aigroove

기능을 구현하는 데서 끝내지 않고, 실제 운영 환경에서 예상하지 못한 요청과 변경에도 안정적으로 동작하는지 검증하는 개발자를 지향합니다.

AI가 음원을 분석해 리듬게임 맵을 자동 생성하는 플랫폼의 관리자 백오피스를 **단독 설계·구현**했고, 인증·인가 체계, 사용자·콘텐츠 관리, AI 모델 운영 UI까지 백엔드 API와 React 프론트엔드를 함께 담당했습니다. 아래 **문제 해결 요약**의 다섯 건은 모두 졸업 프로젝트 **종료 후** 개인 저장소에서 코드를 다시 검증하며 발견하고 고친 것입니다.

프로젝트 개요

기간	2025.03.03 ~ 2025.06.06 (약 3개월)	팀 구성	4인 (가천대학교 종합프로젝트)
담당	관리자 백오피스 풀스택 — 백엔드 API·서비스 + React 프론트엔드		
진행 경과	2025.03 ~ 06 졸업 프로젝트(4인) — 관리자 백오피스 설계·구현 및 학과 서버 배포 2026.06 ~ 08 개인 리팩토링 저장소 — 인가 결함 수정, 통합 테스트 도입, 쿼리 최적화, AWS 운영 전환		
규모	REST API 33개 · Controller 13 · Service 9 · DTO 15 · JPA Entity 15 · React 20 페이지		
Backend	Java 17, Spring Boot 3.4.4, Spring Security, Spring Data JPA, jjwt 0.11.5, MariaDB, Swagger, Gradle		
Frontend	React 18.2, React Router 6, Axios		
배포	React 빌드를 Spring Boot static에 통합한 단일 JAR을 AWS EC2(Ubuntu 24.04)에 배포. systemd 서비스로 등록해 장애 시 자동 재시작·부팅 시 기동을 보장하고, Nginx 리버스 프록시와 Let's Encrypt HTTPS(certbot 자동 갱신)를 적용해 도메인으로 운영.		

문제 해결 요약 종료 후

다섯 건 모두 졸업 프로젝트 종료 후 개인 저장소에서 진행했습니다. 팀 개발 당시에는 드러나지 않았거나, 드러났어도 다시 확인하지 않아 남아 있던 문제들입니다.

발견	원인	조치	검증
게임 유저 토큰으로 <code>/admin/dashboard</code> 가 200 응답 — 통계·로그 노출	인가 규칙이 <code>.authenticated()</code> 뿐이라 권한은 검사하지 않음	JWT에 <code>type</code> 클레임 추가, <code>.hasRole("ADMIN")</code> 으로 전환	인증·인가 통합 테스트 5건 + 조건을 되돌려 실패하는지 역검증
목록 조회 4곳에서 N+1 쿼리	연관 엔티티 Lazy Loading이 행마다 반복 조회	JPQL Fetch Join 적용, <code>@ManyToOne</code> 8곳 LAZY 명시	1 + N회 → 1회
대시보드 집계가 데이터 증가에 비례해 메모리 사용	전체 User를 로드해 Java Stream으로 합산	<code>SUM</code> 단일 집계 쿼리로 전환	로드 건수와 무관한 상수 메모리
HTTPS 전환 후 same-origin 로그인인 CORS 403	프록시 뒤에서 scheme을 http로 인식해 교차 출처로 오판	X-Forwarded-Proto를 신뢰하도록 전환	요청을 재현해 403 소멸 확인
장애 시 무엇이 바뀌었는지 확인 불가 — 원인 추적에 이틀	배포가 형상관리 밖(폴더 통째 교체)에 있어 비교할 기준이 없음	브랜치·PR 도입, 배포를 systemd 관리 서비스로 전환	모든 변경이 diff로 남음

영역	범위	담당
Admin Frontend	React 20페이지 · 공통 컴포넌트 10 · Axios 인터셉터	단독
Admin Backend	Controller 13 · Service 9 · DTO 15 (REST API 33개)	단독
인증 · 인가	SecurityConfig · JWT 필터	공동
공통 도메인	Entity 15 · Repository 14	단독
AI 연동 계층	Controller 3 · 학습 요청 JSON 스키마 설계	단독 *
AI 학습 서비스	모델 학습 로직 · Python 스크립트	팀원
게임 서버 · 클라이언트	멀티플레이 API · Unreal Engine	팀원

* 학습 요청 JSON 스키마(TrainingRequestDTO , TrainingParams)는 AI 담당 팀원과 협의해 설계했습니다.

주요 화면

- 대시보드 — 4개 Repository를 조합한 KPI 6종 집계와 시스템 로그
- AI 학습 실행 — 하이퍼파라미터 6종 설정 및 1초 간격 진행률 polling
- 관리자 승인 — 승인 대기 분리 표시 및 3단계 역할 부여

핵심 구현 ① 인가 검증 누락으로 인한 권한 상승 발견·수정 **종료 후**

면접에서 JWT 구현에 대한 질문을 받은 뒤 코드를 다시 검토하는 과정에서, 인증 필터가 실제로는 동작하지 않는 상태를 확인했습니다. 이를 수정하고 직접 요청을 보내 검증하던 중 **게임 유저 토큰으로도 /admin/dashboard 가 200을 반환하는 것**을 발견했습니다. 사용자 통계와 시스템 로그가 그대로 노출되는 상태였습니다.

원인 — 세 가지가 겹쳐 있었습니다

- **username 중복 가능** — admin 과 user 는 별도 테이블이고 각자 중복 검사만 하므로, 공개 API인 게임 회원가입으로 관리자와 같은 이름을 만들 수 있었습니다.
- **발급 주체 구분 불가** — 두 로그인인 발급하는 JWT 형식이 같아 토큰만으로는 어느 쪽에서 나왔는지 알 수 없었습니다.
- **인가 조건이 인증만 검사** — /admin/** 에 .authenticated() 만 걸려 있어, 이름이 겹치지 않아도 인증된 게임 유저면 통과했습니다.

.authenticated() 는 "인증된 사용자인가"만 검사하고 어떤 권한인지는 보지 않습니다. 인증(Authentication)과 인가(Authorization)를 구분하지 않은 것이 근본 원인이었습니다.

해결 — 토큰에 발급 주체를 담고, 인가 조건을 권한 기반으로 전환

- JWT 발급 시 type 클레임(ADMIN / USER)을 기록 — 서명에 포함되므로 위조 불가
- 필터가 토큰이 가리키는 테이블에서만 조회하도록 변경 — 이름이 겹쳐도 교차 인증되지 않음
- 인가 조건을 .hasRole("ADMIN") 으로 전환

요청	수정 전	수정 후
토큰 없이 /admin/dashboard	403	401
게임 유저 토큰	200 (데이터 노출)	403
관리자 토큰	403	200

회귀 방지 — 인증·인가 통합 테스트 5건

인증·인가는 필터·시큐리티 설정·컨트롤러가 함께 동작해야 결과가 결정되므로, 단위 테스트로는 검증이 어렵다고 판단해 @SpringBootTest + MockMvc 통합 테스트로 작성했습니다. @BeforeEach 에서 관리자와 게임 유저에게 **같은 username**을 부여해, 실제로 권한 상승이 가능했던 조건을 테스트 환경에 재현해 두었습니다.

테스트가 유효한지 역검증 — 작성 후 hasRole("ADMIN") 을 authenticated() 로 되돌린 상태에서 실행해 **권한 부족 테스트만 실패하는 것**을 확인한 뒤 원복했습니다. 통과만 보고는 그 테스트가 무엇을 검증하는지 알 수 없다고 판단했기 때문입니다. 실제로 초기 작성 시 요청에 토큰을 싣는 코드를 빠뜨렸는데도 테스트가 통과한 경우가 있었습니다.

핵심 구현 ② JWT 기반 Stateless 인증 구조

①의 수정을 반영한 현재 인증 구조입니다. 관리자 승인 프로세스는 졸업 프로젝트 당시 설계한 도메인 로직입니다.

- **인증과 인가의 역할 분리** — 필터는 인증만 담당합니다. 토큰이 유효하면 `SecurityContextHolder` 에 권한을 저장하고, 유효하지 않으면 아무것도 하지 않은 채 다음 필터로 넘깁니다. 접근 거부 판단은 `SecurityConfig` 의 인가 규칙입니다. 필터가 직접 응답을 만들면 공개 경로 목록을 두 곳에서 관리하게 되어 서로 어긋날 수 있기 때문입니다.
- **상태 코드 구분** — `authenticationEntryPoint` 를 지정해 인증 정보가 없으면 401, 권한이 부족하면 403을 반환합니다. (Spring Security 기본값은 두 경우 모두 403)
- **관리자 승인 프로세스** — 가입 시 `signupDate = null` 로 저장되고, 기존 관리자(MASTER)가 승인해야 로그인 가능합니다. 무분별한 관리자 계정 생성을 차단하는 도메인 로직입니다.

성능 최적화 종료 후

JPA N+1 문제 해결

목록 조회 API에서 Lazy Loading으로 인한 N+1을 확인하고 JPQL Fetch Join으로 최적화했습니다. `@ManyToOne` 8곳은 전부 `fetch = LAZY` 를 명시했습니다.

대상	관계	최적화 전	최적화 후
로그 목록	<code>Log</code> → <code>Admin</code> / <code>User</code>	1 + N회	1회
문의 목록	<code>Inquiry</code> → <code>User</code> / <code>Admin</code>	1 + N회	1회
공지 목록	<code>Notice</code> → <code>Admin</code>	1 + N회	1회
데이터셋 목록	<code>DatasetInfo</code> → <code>Admin</code>	1 + N회	1회

대시보드 집계 쿼리 최적화

총 곡 업로드 수 계산에서 전체 User를 메모리에 로드해 Java Stream으로 합산하던 로직을, `SELECT COALESCE(SUM(u.uploadedSongCount), 0)` 단일 집계 쿼리로 전환했습니다. 데이터가 늘어날수록 애플리케이션 메모리 사용량이 선형으로 증가하는 구조였기 때문입니다.

트러블슈팅 ① Spring Security + React SPA 통합 배포 졸업 프로젝트

React 빌드를 Spring Boot에 통합해 배포하자, `/admin/dashboard` 같은 클라이언트 라우트 접속 시 인증 오류가 발생하고 새로고침하면 404가 반환됐습니다.

원인	해결
Security가 React 라우팅 경로를 API 엔드포인트로 인식	<code>SecurityConfig</code> 에서 정적 리소스와 공개 경로를 인가 규칙에 명시
새로고침 시 서버에 실제 리소스가 없어 404	<code>WebController</code> 에서 확장자 없는 경로만 정규식으로 매칭해 <code>index.html</code> 로 forward

처음에는 `WebConfig` 의 `addViewController` 로 시도했으나 정적 파일까지 포워딩되는 문제가 있어, 별도 `@Controller` 로 분리하고 정규식으로 확장자 있는 경로를 제외했습니다. 이후 역할이 없어진 `WebConfig` 는 제거하고 CORS 설정은 `SecurityConfig` 로 통합했습니다.

트러블슈팅 ② 운영 환경 전환에서 드러난 결함 4건 종료 후

학과 서버에서 AWS로 옮기고 도메인-HTTPS를 적용하는 과정에서, 로컬과 단일 서버 환경에서는 드러나지 않던 결함 4건이 나타났습니다. 네 건 모두 코드가 아니라 코드가 서 있던 전제가 바뀌면서 드러난 사례였습니다.

증상	원인	해결
로그인 후 모든 관리자 API가 401 — 화면이 로딩에서 멈춤	컴포넌트 20개 파일 30여 곳이 토큰 주입용 axios 인스턴스 대신 순수 axios 를 호출. 기존 permitAll 설정 때문에 토큰 없이도 통과해 결함이 가려져 있었음	30여 곳을 고치는 대신 진입점에 전역 요청 인터셉터를 등록해 일괄 주입. 파일 1개 수정으로 해결돼 기존 코드 회귀 위험이 없음
HTTPS 적용 후 same-origin 로그인 요청이 CORS 403	Nginx가 TLS를 종료하고 평문 HTTP로 전달해 Spring이 scheme을 http 로 인식. Origin 의 https 와 어긋나 교차 출처로 오판	server.forward-headers-strategy=native 로 X-Forwarded-Proto 를 신뢰하도록 전환
/admin/** 페이지 새로고침 시 401	SPA 라우트와 REST API가 모두 /admin/ 으로 시작. 브라우저 페이지 요청에는 JWT 헤더가 붙지 않아 index.html 이 내려가지 못하고 React가 부팅조차 하지 못함	authenticationEntryPoint 에서 Accept 헤더로 페이지 요청과 API 요청을 분기
Swagger 문서가 비어 있음	springdoc.pathsToMatch 가 /game/** 로 지정돼 실제 경로 /api/game/** 와 불일치	설정값을 실제 매핑에 맞춰 수정하고, 재발을 막는 주석을 남김

첫 번째 건은 인가를 `hasRole("ADMIN")` 으로 잠그자 숨어 있던 미부착 호출이 한꺼번에 표면화된 것으로, **보안을 강화하는 작업이 기존 코드의 결함을 드러낸 사례**입니다. 로컬에서는 프론트와 백엔드가 다른 포트로 분리돼 있어 재현되지 않았습니다. 세 번째 건은 트리블슈팅 ①의 SPA 라우팅 문제의 연장으로, 당시에는 공개 경로만 예외 처리했으나 인증이 필요한 경로에는 같은 문제가 남아 있었습니다.

개발 방식 개선 종료 후

졸업 프로젝트 당시 배포는 **SSH로 학과 서버에 접속해 담당 폴더를 통째로 교체하는 방식**이었습니다. 빠르지만 배포된 코드가 저장소를 거치지 않아, 장애가 나도 무엇이 달라졌는지 **diff로 비교할 수 없었습니다**. 실제로 이전 버전에서 동작하던 빌드가 새 버전에서 실패했을 때 권한·경로·설정을 하나씩 바꿔보며 이틀을 썼습니다. 문제는 난이도가 아니라 비교할 기준이 없다는 것이었고, 앞의 인가 검증 누락이 오래 남은 것도 같은 뿌리였습니다.

이후 개인 저장소에서는 모든 변경이 이력으로 남도록 **기능 단위 브랜치와 PR**을 적용하고, PR 본문에 변경 의도와 검증 방법을 남겨 병합 전 자신의 diff를 다시 읽는 절차로 쓰고 있습니다. 배포도 수작업 폴더 교체에서 **systemd 관리 서비스**로 옮겨, 실행 주체·환경변수·재시작 정책이 누군가의 기억이 아니라 설정 파일에 남도록 했습니다.

AI 도구 활용

AI에 코드 생성을 맡기되, **생성된 결과를 그대로 신뢰하지 않는 검증 루프**를 두고 작업했습니다. 설정이 붙어 있어도 실제로 동작하지 않는 경우가 있어 보안 관련 코드는 반드시 직접 요청을 보내 확인했고, 수정 후에는 조건을 되돌려 테스트가 실패하는지까지 검증했습니다. 시크릿은 설정 파일에 `${ENV_VAR}` 플레이스홀더로 두고 실제 값은 `systemd EnvironmentFile` 로 주입해, 설정 파일 자체를 형상관리에 포함할 수 있게 했습니다.

한계와 다음 개선 과제

- `admin.username` / `user.username` 에 DB 유니크 제약이 없어 중복 방지가 애플리케이션 코드에만 의존합니다. 운영 중 발생했던 `NonUniqueResultException` 도 같은 뿌리로 보고 있으며, 유니크 인덱스 추가를 우선 과제로 두고 있습니다.
- 테스트가 인증·인가 영역에 한정되어 있습니다. 서비스 계층 단위 테스트로 범위를 넓히는 것이 다음 순서입니다.
- 관리자 역할이 `MASTER` / `USER` / `AI` 세 가지로 정의되어 있으나 인가 규칙은 관리자 여부만 검사합니다. 역할별 권한 분리(RBAC)를 적용해야 읽기 전용 데모 계정을 공개할 수 있어, 이를 다음 과제로 두고 있습니다.
- AWS 재배포로 상시 기동 환경은 확보했습니다. 다만 공개 데모로 열기에는 시드 데이터와 역할 기반 접근 제어가 남아 있어, 현재는 저장소의 화면 캡처로 갈음하고 있습니다.

개인 리팩토링 저장소 github.com/kimhyeongjun-1204/aigroove (백엔드) · github.com/kimhyeongjun-1204/aigroove-admin (프론트엔드)

원본 팀 저장소 github.com/mys990930/aigroove — 팀 개발 당시의 원본이며, 개인 저장소는 담당 영역을 정리하고 리팩토링을 이어가는 저장소입니다.